

Advanced Java Programming — Complete Cheatsheet

Unit 1: Programming in Java

1.1 Java Architecture, Buzzwords, Path & ClassPath

Java Architecture: - **JDK** (Java Development Kit) = JRE + development tools (javac, jar, etc.) - **JRE** (Java Runtime Environment) = JVM + libraries - **JVM** (Java Virtual Machine) — executes bytecode, platform-independent

Process: Source (`.java`) → `javac` → Bytecode (`.class`) → `java` → JVM → Machine code

Bytecode: Platform-independent intermediate code understood by JVM.

JVM Architecture: Class Loader → Bytecode Verifier → Execution Engine (Interpreter + JIT) → Memory Areas (Heap, Stack, Method Area, PC Registers, Native Method Stack)

Java Buzzwords (write any 3-4 in exam): - **Simple** — syntax based on C/C++, no pointers, no operator overloading - **Object-Oriented** — everything except primitives is an object - **Platform Independent** — bytecode runs on any JVM - **Secure** — no explicit pointers, bytecode verification, sandbox - **Robust** — strong memory management, exception handling, type checking - **Multithreaded** — built-in support for concurrent programming - **Portable** — same bytecode on any platform - **Dynamic** — runtime class loading, reflection

Path vs ClassPath:

Path	ClassPath
System variable for OS to find executables (<code>javac</code> , <code>java</code>)	Tells JVM where to find <code>.class</code> files and libraries
Set to <code><JDK>/bin</code> directory	Set to directories or JAR files containing classes
Used by OS to locate commands	Used by JVM/compiler to locate dependencies

Setting: `javac -cp .;lib/* MyProgram.java` OR `export CLASSPATH=/home/lib`

Compiling and Running:

```
// Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

```
javac Hello.java      # creates Hello.class
java Hello            # runs the program
java Hello arg1 arg2 # with command-line args
```

The `main` method signature: `public static void main(String[] args)` — JVM calls this.

1.2 Arrays, for-each, Class & Object, OOP Features

Arrays:

```
int[] arr = new int[5];
int[] arr2 = {1, 2, 3, 4, 5};
int[][] matrix = new int[3][3];

// Length
arr.length; // NOT a method — it's a property

// Copying
System.arraycopy(src, srcPos, dest, destPos, length);
Arrays.copyOf(original, newLength);
```

for-each loop (enhanced for):

```
int[] numbers = {1, 2, 3, 4, 5};
for (int num : numbers) {
    System.out.println(num);
}
// Cannot modify array elements — use traditional for for that
```

Class and Object:

```
public class Student {
    // Instance variables
    private int id;
    private String name;
```

```
// Constructor – same name as class, no return type
public Student(int id, String name) {
    this.id = id;
    this.name = name;
}

// Methods
public void display() {
    System.out.println(id + ": " + name);
}

// Getter/Setter
public int getId() { return id; }
public void setId(int id) { this.id = id; }
}

// Usage
Student s = new Student(1, "Ram");
s.display();
```

Constructor types: Default (no-arg), Parameterized, Copy constructor (via passing object)

this keyword: Refers to current object, used to distinguish instance variables from parameters.

Method Overloading (Compile-time Polymorphism):

```
class MathUtils {
    public int add(int a, int b) { return a + b; }
    public int add(int a, int b, int c) { return a + b + c; }
    public double add(double a, double b) { return a + b; }
}

// Same name, different parameters (type/count/order). Return type alone is NOT enough.
```

Access Privileges (Access Modifiers):

Modifier	Class	Package	Subclass	World
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default (none)	✓	✓	✗	✗
private	✓	✗	✗	✗

Interface:

```
// Interface – 100% abstract (prior to Java 8)
interface Drawable {
    void draw(); // implicitly public abstract
    default void paint() { } // Java 8+ default method
    static int getVersion() { return 1; } // Java 8+ static method
}

class Circle implements Drawable {
    @Override
    public void draw() {
        System.out.println("Drawing circle");
    }
}

// A class can implement MULTIPLE interfaces
// interface can EXTEND multiple interfaces
```

Inner Class:

```
class Outer {
    private int x = 10;

    // Member Inner Class
    class Inner {
        void show() { System.out.println(x); }
    }

    // Static Nested Class
    static class StaticNested {
        void display() { System.out.println("Static nested"); }
    }
}
```

```
// Local Inner Class (inside method)
void method() {
    class Local { void print() { System.out.println("Local"); } }
    new Local().print();
}

// Anonymous Inner Class
void test() {
    Runnable r = new Runnable() {
        public void run() {
            System.out.println("Anonymous");
        }
    };
}

// Usage: Outer.Inner obj = new Outer().new Inner();
```

final modifier: - final class — cannot be extended (e.g., String, Math) - final method — cannot be overridden - final variable — cannot be reassigned (constant)

```
final int MAX = 100;
// MAX = 200; // ERROR!

final class FinalClass { }
// class Child extends FinalClass { } // ERROR!
```

static modifier: - static variable — one copy shared across all objects (class-level) - static method — belongs to class, can only access static members - static block — runs once when class is loaded - static import — import static members directly

```
class Counter {
    static int count = 0; // class variable
    int instanceCount = 0; // instance variable

    static { System.out.println("Static block runs once"); }

    Counter() {
        count++;
        instanceCount++;
    }

    static void showCount() {
        System.out.println("Total: " + count);
        // System.out.println(instanceCount); // ERROR - can't access instance
    }
}
```

Packages:

```
package com.example; // must be first line

import java.util.Scanner; // single import
import java.util.*; // wildcard import
import static java.lang.Math.PI; // static import
```

- Package = namespace / directory structure
- `javac -d . MyClass.java` creates directory structure matching package
- Fully qualified name: `java.util.Scanner`

Inheritance:

```
class Animal {
    protected String name;
    public void eat() { System.out.println("Eating..."); }
}

class Dog extends Animal { // single inheritance
    public void bark() { System.out.println("Barking..."); }
}

// Java does NOT support multiple inheritance of classes
// Use interfaces for multiple inheritance of type
```

super keyword:

```
class Parent {
    Parent() { System.out.println("Parent"); }
    void show() { System.out.println("Parent show"); }
```

```

}

class Child extends Parent {
    Child() { super(); } // calls parent constructor – implicitly added
    void show() {
        super.show(); // calls parent method
        System.out.println("Child show");
    }
}

```

Method Overriding (Runtime Polymorphism):

```

class Bank {
    double getRate() { return 0; }
}

class NBB extends Bank {
    @Override
    double getRate() { return 5.5; } // same signature
}

class NIC extends Bank {
    @Override
    double getRate() { return 6.0; }
}

// Polymorphism: Bank b = new NBB(); b.getRate(); // => 5.5

```

Rules: same name, same params, same/co-variant return, not weaker access.

Abstract class:

```

abstract class Shape {
    abstract void draw(); // no body – must be overridden
    void display() { System.out.println("Shape"); } // concrete method
}

class Circle extends Shape {
    void draw() { System.out.println("Drawing circle"); }
}

// Cannot instantiate abstract class: new Shape() – ERROR

```

1.3 Exception Handling

Exception Hierarchy:

```

Throwable
├── Error (unrecoverable – OutOfMemoryError, StackOverflowError)
└── Exception
    ├── RuntimeException (unchecked – NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException)
    └── Checked Exception (IOException, SQLException, ClassNotFoundException)

```

Try-Catch-Finally:

```

try {
    int result = 10 / 0; // throws ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage());
} catch (Exception e) { // multi-catch: catch specific to general
    e.printStackTrace();
} finally {
    // ALWAYS executes (whether exception occurs or not)
    // Used for cleanup: closing files, DB connections
    System.out.println("Finally block");
}

```

Java 7+ multi-catch:

```

try {
    // code
} catch (IOException | SQLException e) {
    // handle both
}

```

Try-with-resources (Java 7+):

```

try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    String line = br.readLine();
} catch (IOException e) {
    e.printStackTrace();
}

```

```
}
// Resource automatically closed – no finally needed
```

throws VS throw:

throws	throw
Inside method body	In method signature
Actually throws an exception	Declares that method may throw exception
<code>throw new IOException("error");</code>	<code>void read() throws IOException { }</code>
Only one exception at a time	Can list multiple: <code>throws IOException, SQLException</code>

```
void validateAge(int age) throws IllegalArgumentException {
    if (age < 0) {
        throw new IllegalArgumentException("Age cannot be negative");
    }
}
```

Checked vs Unchecked: - **Checked:** Must handle (try-catch or throws) — `IOException`, `SQLException`, `ClassNotFoundException` -

Unchecked: `RuntimeException` subclasses — no handling required — `NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`, `NumberFormatException`

finally block behavior: - Executes even if `return` is in try/catch - Does NOT execute if `System.exit()` is called in try - Does NOT execute if JVM crashes

Creating Custom Exception:

```
// Checked custom exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message); // pass to parent Exception
    }
}

// Unchecked custom exception
class InvalidAgeException extends RuntimeException {
    public InvalidAgeException(String message) {
        super(message);
    }
}

// Usage
void setAge(int age) throws InvalidAgeException {
    if (age < 0 || age > 150) {
        throw new InvalidAgeException("Invalid age: " + age);
    }
}
```

1.4 Concurrency (Multithreading)

Creating threads — two ways:

```
// 1. Extending Thread class
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running: " + Thread.currentThread().getName());
    }
}
new MyThread().start(); // DON'T call run() directly

// 2. Implementing Runnable (preferred – can extend another class)
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread");
    }
}
new Thread(new MyRunnable()).start();

// Lambda (Java 8+)
new Thread(() -> System.out.println("Lambda thread")).start();
```

Thread States (Life Cycle):

NEW → RUNNABLE → RUNNING → BLOCKED/WAITING → RUNNABLE → TERMINATED

1. **NEW** — `Thread t = new Thread()` created but not started
2. **RUNNABLE** — `t.start()` called, ready to run
3. **RUNNING** — JVM picks it, `run()` executes
4. **BLOCKED** — waiting for monitor lock (synchronization)
5. **WAITING** — `wait()`, `join()`, `sleep()` — waiting indefinitely
6. **TIMED_WAITING** — `sleep(ms)`, `wait(ms)`, `join(ms)` — waiting with timeout
7. **TERMINATED** — `run()` completes

Thread Methods:

```
Thread.currentThread()    // get current thread reference
t.getName() / t.setName() // name
t.getPriority()           // 1 (MIN) to 10 (MAX), default 5 (NORM)
t.sleep(2000)             // static - pauses current thread for ms
t.join()                  // waits for t to die
t.yield()                 // hint to scheduler: give other threads a chance
t.setDaemon(true)         // daemon thread - JVM exits when only daemon threads remain
```

Thread Synchronization:

Problem: When multiple threads access shared data, race conditions occur.

```
class Counter {
    private int count = 0;

    // 1. Synchronized method
    public synchronized void increment() {
        count++;
    }

    // 2. Synchronized block (finer control)
    public void decrement() {
        synchronized (this) {
            count--;
        }
    }
}
```

`synchronized` locks on an object's intrinsic lock (monitor).

Inter-thread Communication (wait-notify):

```
class SharedResource {
    private int data;
    private boolean available = false;

    public synchronized void produce(int value) {
        while (available) {
            wait(); // releases lock, waits for notification
        }
        data = value;
        available = true;
        notify(); // wakes one waiting thread
    }

    public synchronized int consume() {
        while (!available) {
            wait();
        }
        available = false;
        notify();
        return data;
    }
}
```

`wait()` — `notify()` — `notifyAll()` :- `wait()` — called inside synchronized block, releases lock, thread waits - `notify()` — wakes ONE random waiting thread - `notifyAll()` — wakes ALL waiting threads - All three must be called from synchronized context

Producer-Consumer with BlockingQueue:

```
import java.util.concurrent.ArrayBlockingQueue;
BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(10);
queue.put(1); // blocks if full
int val = queue.take(); // blocks if empty
```

Thread Priorities:

```
t.setPriority(Thread.MAX_PRIORITY); // 10
t.setPriority(Thread.MIN_PRIORITY); // 1
t.setPriority(Thread.NORM_PRIORITY); // 5 (default)
```

Note: Priority is a hint — OS scheduler decides actual execution.

1.5 Working with Files (I/O)

Byte Stream Classes (binary):

```
// Writing bytes
FileOutputStream fos = new FileOutputStream("file.dat");
fos.write(65); // writes 'A'
fos.write("Hello".getBytes());
fos.close();

// Reading bytes
FileInputStream fis = new FileInputStream("file.dat");
int ch;
while ((ch = fis.read()) != -1) {
    System.out.print((char) ch);
}
fis.close();
```

Character Stream Classes (text):

```
// Writing text
FileWriter fw = new FileWriter("file.txt");
fw.write("Hello World");
fw.close();

// Reading text
FileReader fr = new FileReader("file.txt");
int ch;
while ((ch = fr.read()) != -1) {
    System.out.print((char) ch);
}
fr.close();
```

Buffered Streams (efficient):

```
BufferedReader br = new BufferedReader(new FileReader("file.txt"));
String line;
while ((line = br.readLine()) != null) {
    System.out.println(line);
}
br.close();

BufferedWriter bw = new BufferedWriter(new FileWriter("file.txt"));
bw.write("Hello");
bw.newLine();
bw.write("World");
bw.close();
```

Using try-with-resources (auto close):

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
```

Byte vs Character Streams:

Byte Stream	Character Stream
<code>InputStream</code> / <code>OutputStream</code>	<code>Reader</code> / <code>Writer</code>
Reads/writes bytes (8-bit)	Reads/writes characters (16-bit Unicode)
<code>FileInputStream</code> , <code>FileOutputStream</code>	<code>FileReader</code> , <code>FileWriter</code>
<code>BufferedInputStream</code>	<code>BufferedReader</code>
For binary data (images, audio, .class)	For text data

Random Access File:

```

RandomAccessFile raf = new RandomAccessFile("file.dat", "rw");
raf.seek(10);           // move to position 10
raf.writeInt(100);       // write at current position
raf.seek(0);             // go back to start
int value = raf.readInt(); // read from current position
System.out.println(raf.getFilePointer()); // current position
raf.close();

```

Modes: "r" (read), "rw" (read+write), "rwd" (sync data), "rws" (sync data+metadata)

Reading and Writing Objects (Serialization):

```

class Employee implements Serializable {
    private static final long serialVersionUID = 1L; // version control
    private int id;
    private String name;
    private transient double salary; // NOT serialized

    // constructor, getters, setters
}

// Writing object
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("emp.dat"));
oos.writeObject(new Employee(1, "Ram"));
oos.close();

// Reading object
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("emp.dat"));
Employee e = (Employee) ois.readObject();
ois.close();

```

Serializable interface: Marker interface (no methods). Only non-static, non-transient fields are serialized.

File class:

```

File file = new File("test.txt");
file.exists(); file.isFile(); file.isDirectory();
file.getName(); file.getAbsolutePath(); file.length();
file.delete(); file.mkdir(); file.mkdirs();
String[] files = file.list(); // list directory contents

```

Unit 2: User Interface Components with Swing

2.1 AWT vs Swing, Applets, Swing Hierarchy

AWT (Abstract Window Toolkit): - Platform-dependent (uses native OS components) - Heavyweight components - Limited set of components - `java.awt` package

Swing: - Platform-independent (painted by Java) - Lightweight components (except top-level) - Richer component set - `javax.swing` package - Follows MVC pattern

AWT	Swing
Heavyweight (native peers)	Lightweight (Java paints)
Platform-dependent look	Platform-independent (Pluggable L&F)
Fewer components	Rich component set
<code>java.awt.*</code>	<code>javax.swing.*</code>
Not all components are Java beans	All are Java beans
No double buffering	Double buffered (smooth)

Swing Class Hierarchy:

```

Object
├── Component (AWT)
│   ├── Container
│   │   ├── Window
│   │   │   ├── Frame → JFrame
│   │   │   └── Dialog → JDialog
│   │   └── Panel → JPanel
│   └── JComponent (Swing base for all lightweight components)
│       ├── JLabel, JButton, JTextField, JTextArea
│       ├── JComboBox, JList, JSlider
│       └── JCheckBox, JRadioButton

```



```

├─ JMenuBar, JMenu, JMenuItem
├─ JTable, JTree
├─ JScrollPane, JTabbedPane
└─ JInternalFrame

```

Component vs Container: - **Component:** Abstract superclass for all UI objects (buttons, labels, text fields) - **Container:** Component that can hold other components (panels, frames). Has `add()` method.

Java Applets (historical):

```

import java.applet.Applet;
import java.awt.Graphics;

public class MyApplet extends Applet {
    public void init() { /* called once when applet loads */ }
    public void start() { /* called after init, also on revisit */ }
    public void stop() { /* called when leaving page */ }
    public void destroy() { /* called when applet ends */ }
    public void paint(Graphics g) {
        g.drawString("Hello", 50, 50);
    }
}

```

Applet Life Cycle: `init()` → `start()` → `paint()` → `stop()` → `destroy()` (Applets are deprecated — not used in modern Java. May still appear as theory.)

Creating a JFrame (basic):

```

import javax.swing.*;

public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame("My App");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

```

2.2 Layout Management

No Layout (Absolute Positioning):

```

frame.setLayout(null);
JButton btn = new JButton("Click");
btn.setBounds(50, 50, 100, 30); // x, y, width, height
frame.add(btn);

```

FlowLayout:

```

frame.setLayout(new FlowLayout()); // center, hgap=5, vgap=5
frame.setLayout(new FlowLayout(FlowLayout.LEFT, 10, 10));
// Components placed left-to-right, wraps to next line
// Default for JPanel

```

BorderLayout:

```

frame.setLayout(new BorderLayout()); // default for JFrame
frame.add(new JButton("North"), BorderLayout.NORTH);
frame.add(new JButton("South"), BorderLayout.SOUTH);
frame.add(new JButton("East"), BorderLayout.EAST);
frame.add(new JButton("West"), BorderLayout.WEST);
frame.add(new JButton("Center"), BorderLayout.CENTER); // default position

```

Regions: NORTH, SOUTH, EAST, WEST, CENTER. Respects component's preferred size in N/S (height) and E/W (width). CENTER gets remaining space.

GridLayout:

```

frame.setLayout(new GridLayout(3, 2)); // 3 rows, 2 columns
frame.setLayout(new GridLayout(3, 2, 10, 10)); // rows, cols, hgap, vgap
// All cells equal size – components fill cell completely

```

GridBagLayout (most powerful):

```

frame.setLayout(new GridBagLayout());
GridBagConstraints gbc = new GridBagConstraints();

gbc.gridx = 0; gbc.gridy = 0; // cell position

```

```

gbc.gridwidth = 2;           // span 2 columns
gbc.gridheight = 1;         // span 1 row
gbc.fill = GridBagConstraints.HORIZONTAL; // fill behavior
gbc.insets = new Insets(5, 5, 5, 5); // padding
gbc.weightx = 1.0; gbc.weighty = 0; // resizing weight
gbc.anchor = GridBagConstraints.WEST; // alignment within cell

frame.add(new JLabel("Name:"), gbc);

```

GroupLayout (NetBeans style — Java 6+):

```

GroupLayout layout = new GroupLayout(panel);
panel.setLayout(layout);
layout.setAutoCreateGaps(true);
layout.setAutoCreateContainerGaps(true);

layout.setHorizontalGroup(
    layout.createSequentialGroup()
        .addComponent(new JLabel("Name:"))
        .addComponent(new JTextField(15))
);

layout.setVerticalGroup(
    layout.createParallelGroup(GroupLayout.Alignment.BASELINE)
        .addComponent(new JLabel("Name:"))
        .addComponent(new JTextField(15))
);

```

Two dimensions: horizontal group (Sequential/Parallel) and vertical group.

2.3 GUI Controls

JTextField:

```

JTextField tf = new JTextField(20); // 20 columns wide
tf.setText("Hello");
String text = tf.getText();
tf.setEditable(false);

```

JPasswordField:

```

JPasswordField pf = new JPasswordField(15);
char[] password = pf.getPassword(); // returns char[] not String (more secure)

```

JTextArea + JScrollPane:

```

JTextArea ta = new JTextArea(5, 20); // rows, columns
ta.setLineWrap(true);
JScrollPane sp = new JScrollPane(ta);
// Always wrap JTextArea in JScrollPane for scrolling

```

JLabel:

```

JLabel label = new JLabel("Enter name:");
label.setIcon(new ImageIcon("icon.png")); // icon + text
label.setHorizontalTextPosition(SwingConstants.RIGHT);

```

JCheckBox:

```

JCheckBox cb = new JCheckBox("Java", true); // text, selected
cb.isSelected();

```

JRadioButton + ButtonGroup:

```

JRadioButton r1 = new JRadioButton("Male", true);
JRadioButton r2 = new JRadioButton("Female");
ButtonGroup bg = new ButtonGroup();
bg.add(r1); bg.add(r2); // only one can be selected at a time

```

JComboBox:

```

String[] items = {"Java", "Python", "C++"};
JComboBox<String> cb = new JComboBox<>(items);
cb.setSelectedIndex(0);
String selected = (String) cb.getSelectedItem();

```

JSlider:

```

JSlider slider = new JSlider(0, 100, 50); // min, max, initial
slider.setMajorTickSpacing(20);
slider.setMinorTickSpacing(5);
slider.setPaintTicks(true);
slider.setPaintLabels(true);

```

Borders:

```

JPanel panel = new JPanel();
panel.setBorder(BorderFactory.createTitledBorder("Personal Info"));
panel.setBorder(BorderFactory.createLineBorder(Color.RED));
panel.setBorder(BorderFactory.createEtchedBorder());
panel.setBorder(BorderFactory.createCompoundBorder(outer, inner));

```

2.4 Menu System

```

JMenuBar menuBar = new JMenuBar();

JMenu fileMenu = new JMenu("File");
JMenu editMenu = new JMenu("Edit");

JMenuItem newItem = new JMenuItem("New");
JMenuItem saveItem = new JMenuItem("Save");
JMenuItem exitItem = new JMenuItem("Exit");

fileMenu.add(newItem);
fileMenu.add(saveItem);
fileMenu.addSeparator(); // horizontal line
fileMenu.add(exitItem);

// Icons in menu items
JMenuItem openItem = new JMenuItem("Open", new ImageIcon("open.png"));

// CheckBox menu item
JCheckBoxMenuItem cbItem = new JCheckBoxMenuItem("Word Wrap");

// RadioButton menu item
JRadioButtonMenuItem rbItem = new JRadioButtonMenuItem("Left Align");

// Keyboard mnemonics & accelerators
fileMenu.setMnemonic('F'); // Alt+F
saveItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_S, KeyEvent.CTRL_DOWN_MASK)); // Ctrl+S
saveItem.setMnemonic('S'); // Alt+F, then S

// Pop-up menu (right-click menu)
JPopupMenu popup = new JPopupMenu();
popup.add(new JMenuItem("Cut"));
popup.add(new JMenuItem("Copy"));
popup.add(new JMenuItem("Paste"));

// Attach to component
component.setComponentPopupMenu(popup);

// Enabling/disabling menu items
saveItem.setEnabled(false);

// Toolbar
JToolBar toolBar = new JToolBar();
toolBar.add(new JButton("Save"));
toolBar.addSeparator();

// Tooltip
button.setToolTipText("Click to save the file");

```

2.5 Dialogs, Internal Frames, Tables, Trees

Option Dialogs:

```

// Message dialog
JOptionPane.showMessageDialog(frame, "File saved", "Success", JOptionPane.INFORMATION_MESSAGE);

```

```
// Confirm dialog
int result = JOptionPane.showConfirmDialog(frame, "Are you sure?", "Confirm",
    JOptionPane.YES_NO_OPTION);

// Input dialog
String name = JOptionPane.showInputDialog(frame, "Enter your name:");

// Custom option dialog
String[] options = {"Blue", "Red", "Green"};
int choice = JOptionPane.showOptionDialog(frame, "Pick a color", "Color Chooser",
    JOptionPane.DEFAULT_OPTION, JOptionPane.PLAIN_MESSAGE, null, options, options[0]);
```

FileChooser:

```
JFileChooser fc = new JFileChooser();
int result = fc.showOpenDialog(frame); // or showSaveDialog
if (result == JFileChooser.APPROVE_OPTION) {
    File file = fc.getSelectedFile();
    String path = file.getAbsolutePath();
}
```

ColorChooser:

```
Color color = JColorChooser.showDialog(frame, "Pick a color", Color.BLUE);
```

JInternalFrame:

```
JInternalFrame internal = new JInternalFrame("Document", true, true, true, true);
// last 4 params: resizable, closable, maximizable, iconifiable
internal.setSize(300, 200);
internal.setVisible(true);
desktopPane.add(internal); // needs JDesktopPane
```

JTable:

```
String[][] data = {{ "1", "Ram", "2", "Shyam" }};
String[] cols = { "ID", "Name" };
JTable table = new JTable(data, cols);
JScrollPane sp = new JScrollPane(table);
```

JTree:

```
DefaultMutableTreeNode root = new DefaultMutableTreeNode("Root");
DefaultMutableTreeNode child1 = new DefaultMutableTreeNode("Child 1");
root.add(child1);
JTree tree = new JTree(root);
```

Unit 3: Event Handling

3.1 Event Handling Concept

Event Delegation Model: - **Source** — component that generates event (button, text field) - **Event** — object representing user interaction (ActionEvent, KeyEvent) - **Listener** — interface that receives and handles the event

Three parts: Event Source → Event Object → Event Listener

Steps to handle events: 1. Implement appropriate Listener interface 2. Register listener with source using `addXxxListener()` 3. Override handler method

ActionEvent example:

```
class MyFrame extends JFrame implements ActionListener {
    JButton btn = new JButton("Click");

    MyFrame() {
        btn.addActionListener(this); // Step 2: register
        add(btn);
    }

    // Step 3: handle
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked: " + e.getActionCommand());
        // e.getActionCommand() returns button text ("Click")
        // e.getSource() returns the button object
        // e.getWhen() returns timestamp
    }
}
```

```
}
}
```

Using ActionCommand:

```
// Multiple buttons – differentiate by action command
JButton saveBtn = new JButton("Save");
JButton delBtn = new JButton("Delete");
saveBtn.setActionCommand("save");
delBtn.setActionCommand("delete");
saveBtn.addActionListener(this);
delBtn.addActionListener(this);

public void actionPerformed(ActionEvent e) {
    switch (e.getActionCommand()) {
        case "save": System.out.println("Saving..."); break;
        case "delete": System.out.println("Deleting..."); break;
    }
}
```

Listener Interfaces vs Adapter Classes:

Listener Interface	Adapter Class
Declares ALL methods of that event type	Provides EMPTY implementations
Must implement every method	Override only needed methods
MouseListener — 5 methods	MouseAdapter — 5 empty methods
More code	Cleaner, less boilerplate

```
// Without adapter – must implement ALL 5 methods even if only need 1
class MyMouseListener implements MouseListener {
    public void mouseClicked(MouseEvent e) { }
    public void mousePressed(MouseEvent e) { }
    public void mouseReleased(MouseEvent e) { }
    public void mouseEntered(MouseEvent e) { }
    public void mouseExited(MouseEvent e) { }
}

// With adapter – only override what you need
class MyMouseAdapter extends MouseAdapter {
    public void mouseClicked(MouseEvent e) {
        System.out.println("Clicked at " + e.getX() + "," + e.getY());
    }
}
```

3.2 Handling Specific Events

ActionEvent:

```
btn.addActionListener(e -> System.out.println("Clicked!"));
```

Triggered by: JButton click, JTextField Enter, JMenuItem select, JCheckBox toggle

KeyEvent:

```
class MyKeyHandler implements KeyListener {
    public void keyPressed(KeyEvent e) {
        int code = e.getKeyCode(); // KeyEvent.VK_ENTER, VK_UP, VK_A
        char ch = e.getKeyChar(); // actual character
        if (code == KeyEvent.VK_ENTER) {
            System.out.println("Enter pressed");
        }
    }
    public void keyReleased(KeyEvent e) { }
    public void keyTyped(KeyEvent e) { }
}

component.addKeyListener(handler);
component.setFocusable(true); // important – or component won't receive key events
```

FocusEvent:

```
class MyFocusHandler implements FocusListener {
    public void focusGained(FocusEvent e) {
        // component received focus
        component.setBackground(Color.YELLOW);
    }
}
```

```

    }

    public void focusLost(FocusEvent e) {
        // component lost focus
        component.setBackground(Color.WHITE);
    }
}

// Use for input validation – validate on focus lost

```

MouseEvent:

```

class MyMouseHandler extends MouseAdapter {
    public void mouseClicked(MouseEvent e) {
        if (e.getClickCount() == 2) {
            System.out.println("Double click at " + e.getX() + ", " + e.getY());
        }
        if (e.getButton() == MouseEvent.BUTTON3) {
            System.out.println("Right click");
        }
    }
    public void mousePressed(MouseEvent e) { }
    public void mouseReleased(MouseEvent e) { }
    public void mouseEntered(MouseEvent e) { }
    public void mouseExited(MouseEvent e) { }
    public void mouseDragged(MouseEvent e) { }
    public void mouseMoved(MouseEvent e) { }
}

```

WindowEvent:

```

class MyWindowHandler extends WindowAdapter {
    public void windowClosing(WindowEvent e) {
        int confirm = JOptionPane.showConfirmDialog(null, "Exit?");
        if (confirm == JOptionPane.YES_OPTION) {
            System.exit(0);
        }
    }
    public void windowOpened(WindowEvent e) { }
    public void windowClosed(WindowEvent e) { }
    public void windowIconified(WindowEvent e) { }
    public void windowDeiconified(WindowEvent e) { }
    public void windowActivated(WindowEvent e) { }
    public void windowDeactivated(WindowEvent e) { }
}

frame.addWindowListener(new MyWindowHandler());
// Or simpler:
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

```

ItemEvent:

```

class MyItemHandler implements ItemListener {
    public void itemStateChanged(ItemEvent e) {
        if (e.getStateChange() == ItemEvent.SELECTED) {
            JComboBox cb = (JComboBox) e.getSource();
            System.out.println("Selected: " + cb.getSelectedItem());
        }
    }
}

// Used by JCheckBox, JRadioButton, JComboBox

```

Unit 4: Database Connectivity (JDBC)

4.1 JDBC Architecture & Driver Types

JDBC Architecture:

```

Java Application
    ↓
JDBC API (java.sql, javax.sql)
    ↓
JDBC Driver Manager
    ↓
JDBC Driver

```

↓
Database

JDBC Driver Types:

Type	Name	Description	Example
1	JDBC-ODBC Bridge	Converts JDBC to ODBC (deprecated)	sun.jdbc.odbc.JdbcOdbcDriver
2	Native API / Part Java	Uses native DB client library	Oracle OCI driver
3	Network Protocol / All Java	Middleware server converts JDBC to DB protocol	IDS Server
4	Thin Driver / All Java	Directly converts JDBC to DB protocol	MySQL Connector/J, Oracle Thin

Type 4 is most common — pure Java, platform-independent, no client installation.

JDBC Configuration Steps:

```
// 1. Load driver (optional since JDBC 4.0 – auto-loaded)
Class.forName("com.mysql.cj.jdbc.Driver");

// 2. Establish connection
String url = "jdbc:mysql://localhost:3306/mydb";
String user = "root";
String pass = "password";
Connection conn = DriverManager.getConnection(url, user, pass);

// 3. Create statement
Statement stmt = conn.createStatement();

// 4. Execute query
ResultSet rs = stmt.executeQuery("SELECT * FROM students");

// 5. Process results
while (rs.next()) {
    int id = rs.getInt("id");           // by column name
    String name = rs.getString(2);      // by column index (1-based)
    System.out.println(id + ": " + name);
}

// 6. Close resources
rs.close(); stmt.close(); conn.close();
```

Managing Connections (Connection Pooling — concept):

```
// In production, use connection pools (HikariCP, Apache DBCP)
// Pool maintains reusable connections – reduces overhead of creating new ones
DataSource ds = new MySQLConnectionPoolDataSource();
// ds.setUrl(), ds.setUser(), ds.setPassword()
Connection conn = ds.getConnection();
```

SQL Exceptions handling:

```
try {
    // JDBC operations
} catch (SQLException e) {
    System.out.println("Error code: " + e.getErrorCode());
    System.out.println("SQL state: " + e.getSQLState());
    System.out.println("Message: " + e.getMessage());
    // SQLException can have chained exceptions
    while ((e = e.getNextException()) != null) {
        System.out.println("Next: " + e.getMessage());
    }
}
```

4.2 DDL, DML, Prepared Statements, Result Sets

DDL Operations (CREATE, ALTER, DROP):

```
Statement stmt = conn.createStatement();
stmt.executeUpdate("CREATE TABLE student ( "
    + "id INT PRIMARY KEY, "
    + "name VARCHAR(50), "
    + "grade CHAR(2))");
```

DML Operations (INSERT, UPDATE, DELETE):

```
// INSERT
int rows = stmt.executeUpdate("INSERT INTO student VALUES (1, 'Ram', 'A+')");

// UPDATE
int rows = stmt.executeUpdate("UPDATE student SET grade='A' WHERE id=1");

// DELETE
int rows = stmt.executeUpdate("DELETE FROM student WHERE id=1");
// executeUpdate returns number of affected rows
```

PreparedStatement (pre-compiled, prevents SQL injection):

```
String sql = "INSERT INTO student (id, name, grade) VALUES (?, ?, ?)";
PreparedStatement pstmt = conn.prepareStatement(sql);

pstmt.setInt(1, 1);          // first ? - index starts at 1
pstmt.setString(2, "Ram");   // second ?
pstmt.setString(3, "A+");    // third ?
pstmt.executeUpdate();

// Batch insert
pstmt.setInt(1, 2); pstmt.setString(2, "Shyam"); pstmt.setString(3, "B");
pstmt.addBatch();
pstmt.setInt(1, 3); pstmt.setString(2, "Hari"); pstmt.setString(3, "A");
pstmt.addBatch();
pstmt.executeBatch();
```

Statement vs PreparedStatement:

Statement	PreparedStatement
Created for each query	Pre-compiled, reusable
SQL injection vulnerable	SQL injection safe (uses escaping)
<code>stmt.executeQuery(sql)</code>	<code>pstmt.executeQuery()</code> — SQL set at creation
Good for DDL	Good for repeated DML
No binary data support	Supports binary data via streams

Scrollable ResultSet:

```
Statement stmt = conn.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE, // scrollable, doesn't reflect DB changes
    ResultSet.CONCUR_READ_ONLY        // read-only
);
ResultSet rs = stmt.executeQuery("SELECT * FROM student");

rs.absolute(3);      // go to row 3
rs.relative(-1);     // go back 1 row
rs.first();          // go to first row
rs.last();           // go to last row
rs.previous();       // go to previous row
rs.beforeFirst();    // before first
rs.afterLast();      // after last
rs.isFirst(); rs.isLast();
```

Updateable ResultSet:

```
Statement stmt = conn.createStatement(
    ResultSet.TYPE_SCROLL_SENSITIVE,
    ResultSet.CONCUR_UPDATABLE
);
ResultSet rs = stmt.executeQuery("SELECT * FROM student");

rs.last();
rs.updateString("name", "Updated Name"); // update field
rs.updateRow();                          // commit to DB

rs.moveToInsertRow();
rs.updateInt("id", 10);
rs.updateString("name", "New");
rs.insertRow();
```



```
rs.absolute(1);
rs.deleteRow();
```

RowSet (disconnected, more flexible than ResultSet):

```
// JdbcRowSet - connected (like ResultSet but scrollable by default)
JdbcRowSet jrs = new JdbcRowSetImpl();
jrs.setUrl("jdbc:mysql://localhost:3306/mydb");
jrs.setCommand("SELECT * FROM student");
jrs.execute();
while (jrs.next()) { /* ... */ }

// CachedRowSet - disconnected (works without DB connection)
CachedRowSet crs = new CachedRowSetImpl();
crs.setUrl("...");
crs.setCommand("SELECT * FROM student");
crs.execute(); // fetches all data, then disconnects
// modify data offline
crs.next();
crs.updateString("name", "Modified");
crs.updateRow();
crs.acceptChanges(conn); // reconnect and sync
```

Transactions:

```
conn.setAutoCommit(false); // disable auto-commit

try {
    stmt.executeUpdate("UPDATE account SET balance = balance - 1000 WHERE id = 1");
    stmt.executeUpdate("UPDATE account SET balance = balance + 1000 WHERE id = 2");
    conn.commit(); // commit both operations
} catch (SQLException e) {
    conn.rollback(); // undo both on error
} finally {
    conn.setAutoCommit(true);
}

// Savepoints (partial rollback)
Savepoint sp = conn.setSavepoint();
conn.rollback(sp); // roll back to savepoint
conn.commit(); // commit remaining
```

Multiple Results (multiple result sets from stored procedure):

```
CallableStatement cs = conn.prepareCall("{call my_proc()}");
boolean hasResult = cs.execute();
while (hasResult) {
    ResultSet rs = cs.getResultSet();
    // process rs
    hasResult = cs.getMoreResults();
}
}
```

SQL Escapes (JDBC-specific syntax):

```
// Date literal
"SELECT * FROM orders WHERE date = {d '2024-12-25'}"

// Time literal
"SELECT * FROM log WHERE time = {t '14:30:00'}"

// Timestamp literal
"SELECT * FROM log WHERE ts = {ts '2024-12-25 14:30:00.0'}"

// Function call
"SELECT {fn UCASE(name)} FROM student"

// Escape for LIKE special characters
String search = "%50%";
String escaped = search.replace("%", "!%").replace("_", "!_");
pstmt.setString(1, escaped);
// Works with: WHERE name LIKE ? {escape '!'}
}
```

Unit 5: Network Programming

5.1 TCP, UDP, Ports, IP, Network Classes

TCP (Transmission Control Protocol): - Connection-oriented - Reliable — guaranteed delivery, in-order - Slower due to overhead - Used for: web (HTTP), email (SMTP), file transfer (FTP)

UDP (User Datagram Protocol): - Connectionless - Unreliable — no guarantee, no ordering - Faster, lower overhead - Used for: video streaming, DNS, online gaming

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable	Unreliable
Ordering	In-order	No ordering
Speed	Slower	Faster
Overhead	Higher (ACK, handshake)	Lower
Use cases	HTTP, FTP, SMTP	DNS, VoIP, streaming

Ports: - 0-1023: Well-known ports (HTTP=80, HTTPS=443, FTP=21, SMTP=25) - 1024-49151: Registered ports - 49152-65535: Dynamic/private ports

IP Address: `InetAddress` class

```
InetAddress local = InetAddress.getLocalHost();
System.out.println(local.getHostName());    // machine name
System.out.println(local.getHostAddress()); // IP address

InetAddress remote = InetAddress.getByName("www.google.com");
System.out.println(remote.getHostAddress());

InetAddress[] all = InetAddress.getAllByName("google.com");
```

5.2 Socket Programming

TCP Socket Programming:

Server:

```
ServerSocket serverSocket = new ServerSocket(8080);
System.out.println("Server waiting on port 8080...");
Socket clientSocket = serverSocket.accept(); // blocking – waits for client

// Communication streams
BufferedReader in = new BufferedReader(
    new InputStreamReader(clientSocket.getInputStream()));
PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

String msg = in.readLine(); // read from client
out.println("Hello from server"); // send to client

clientSocket.close();
serverSocket.close();
```

Client:

```
Socket socket = new Socket("localhost", 8080);

BufferedReader in = new BufferedReader(
    new InputStreamReader(socket.getInputStream()));
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

out.println("Hello from client"); // send to server
String msg = in.readLine();       // read from server

socket.close();
```

Complete TCP Example — Echo Server:

```
// Server
public class EchoServer {
    public static void main(String[] args) throws IOException {
        ServerSocket ss = new ServerSocket(6666);
        while (true) {
            Socket s = ss.accept();
```

```

        DataInputStream dis = new DataInputStream(s.getInputStream());
        DataOutputStream dos = new DataOutputStream(s.getOutputStream());
        String str = dis.readUTF();
        dos.writeUTF("Echo: " + str);
        s.close();
    }
}

// Client
public class EchoClient {
    public static void main(String[] args) throws IOException {
        Socket s = new Socket("localhost", 6666);
        DataOutputStream dos = new DataOutputStream(s.getOutputStream());
        DataInputStream dis = new DataInputStream(s.getInputStream());
        dos.writeUTF("Hello");
        System.out.println(dis.readUTF());
        s.close();
    }
}

```

UDP Socket Programming:

Server:

```

DatagramSocket socket = new DatagramSocket(9876);
byte[] buffer = new byte[1024];
DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
socket.receive(packet); // blocking

String msg = new String(packet.getData(), 0, packet.getLength());
InetAddress clientAddr = packet.getAddress();
int clientPort = packet.getPort();

// Send response
String response = "Received: " + msg;
byte[] respData = response.getBytes();
DatagramPacket respPacket = new DatagramPacket(
    respData, respData.length, clientAddr, clientPort);
socket.send(respPacket);

socket.close();

```

Client:

```

DatagramSocket socket = new DatagramSocket();
String msg = "Hello";
byte[] data = msg.getBytes();
InetAddress addr = InetAddress.getByName("localhost");

DatagramPacket packet = new DatagramPacket(data, data.length, addr, 9876);
socket.send(packet);

// Receive response
byte[] buffer = new byte[1024];
DatagramPacket resp = new DatagramPacket(buffer, buffer.length);
socket.receive(resp);
System.out.println(new String(resp.getData(), 0, resp.getLength()));

socket.close();

```

Working with URLs:

```

URL url = new URL("https://example.com/page?q=hello");
System.out.println(url.getProtocol()); // https
System.out.println(url.getHost()); // example.com
System.out.println(url.getPort()); // -1 (default)
System.out.println(url.getPath()); // /page
System.out.println(url.getQuery()); // q=hello

// Reading from URL
BufferedReader br = new BufferedReader(
    new InputStreamReader(url.openStream()));
String line;

```

```
while ((line = br.readLine()) != null) {
    System.out.println(line);
}
br.close();
```

URLConnection class:

```
URL url = new URL("https://example.com");
URLConnection conn = url.openConnection();
conn.setRequestProperty("User-Agent", "Mozilla/5.0");

// Reading headers
System.out.println(conn.getContentType());
System.out.println(conn.getContentLength());
System.out.println(conn.getLastModified());

// Reading content
BufferedReader br = new BufferedReader(
    new InputStreamReader(conn.getInputStream()));
String line;
while ((line = br.readLine()) != null) {
    System.out.println(line);
}
br.close();
```

5.3 Java Mail API

Sending Email:

```
// 1. Set properties
Properties props = new Properties();
props.put("mail.smtp.host", "smtp.gmail.com");
props.put("mail.smtp.port", "587");
props.put("mail.smtp.auth", "true");
props.put("mail.smtp.starttls.enable", "true");

// 2. Create session with authentication
Session session = Session.getInstance(props,
    new Authenticator() {
        protected PasswordAuthentication getPasswordAuthentication() {
            return new PasswordAuthentication("user@gmail.com", "password");
        }
    });

// 3. Create message
MimeMessage msg = new MimeMessage(session);
msg.setFrom(new InternetAddress("sender@gmail.com"));
msg.setRecipients(Message.RecipientType.TO, "recipient@example.com");
msg.setSubject("Test Subject");
msg.setText("Hello! This is a test email.");

// 4. Send
Transport.send(msg);
```

Sending with attachment:

```
MimeBodyPart textPart = new MimeBodyPart();
textPart.setText("Please find the attachment.");

MimeBodyPart filePart = new MimeBodyPart();
filePart.attachFile("/path/to/file.pdf");

Multipart multipart = new MimeMultipart();
multipart.addBodyPart(textPart);
multipart.addBodyPart(filePart);

msg.setContent(multipart);
Transport.send(msg);
```

Receiving Email (IMAP/POP3):

```
Properties props = new Properties();
props.put("mail.store.protocol", "imaps");
```

```

Session session = Session.getInstance(props);
Store store = session.getStore("imaps");
store.connect("imap.gmail.com", "user@gmail.com", "password");

Folder inbox = store.getFolder("INBOX");
inbox.open(Folder.READ_ONLY);

Message[] messages = inbox.getMessage();
for (Message m : messages) {
    System.out.println("From: " + m.getFrom()[0]);
    System.out.println("Subject: " + m.getSubject());
}

inbox.close();
store.close();

```

Unit 6: GUI with JavaFX

6.1 JavaFX vs Swing, Layouts

JavaFX vs Swing:

Feature	JavaFX	Swing
Release	Java 8+ (replaces Swing)	Java 1.2+
Styling	CSS styling	Look-and-Feel only
Rendering	Hardware-accelerated (Prism)	Software-rendered (AWT)
FXML	Declarative UI via FXML	Programmatic only
Property binding	Built-in observable/binding	Manual listener setup
Animation	Built-in animation API	No built-in animation
Multimedia	Video, audio support	No multimedia
WebView	Embedded web browser	No
Scene graph	Yes — nodes tree	No scene graph

JavaFX Application Structure:

```

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class HelloFX extends Application {
    @Override
    public void start(Stage stage) {
        Button btn = new Button("Click Me");
        btn.setOnAction(e -> System.out.println("Clicked!"));

        StackPane root = new StackPane(btn);
        Scene scene = new Scene(root, 300, 200);
        stage.setTitle("Hello JavaFX");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

JavaFX Life Cycle: `init()` → `start(Stage)` → application running → `stop()`

FlowPane:

```

FlowPane pane = new FlowPane();
pane.setHgap(10); pane.setVgap(10);
pane.setAlignment(Pos.CENTER);
pane.getChildren().addAll(new Label("Name:"), new TextField());

```

```
// Default orientation – horizontal, wraps to next line
```

BorderPane:

```
BorderPane pane = new BorderPane();
pane.setTop(new Label("Header"));
pane.setLeft(new Button("Left"));
pane.setCenter(new TextArea());
pane.setRight(new Button("Right"));
pane.setBottom(new Label("Footer"));
// Regions: Top, Bottom, Left, Right, Center
```

HBox:

```
HBox hbox = new HBox(10); // spacing = 10
hbox.setAlignment(Pos.CENTER);
hbox.getChildren().addAll(new Label("Name:"), new TextField());
// Horizontal arrangement – all in one row
```

VBox:

```
VBox vbox = new VBox(10); // spacing = 10
vbox.setAlignment(Pos.CENTER);
vbox.getChildren().addAll(new Label("Name:"), new TextField(), new Button("Submit"));
// Vertical arrangement – all in one column
```

GridPane (most flexible):

```
GridPane grid = new GridPane();
grid.setHgap(10); grid.setVgap(10);
grid.setPadding(new Insets(20));

GridPane.setConstraints(new Label("Name:"), 0, 0); // col, row
GridPane.setConstraints(new TextField(), 1, 0);
GridPane.setConstraints(new Label("Email:"), 0, 1);
GridPane.setConstraints(new TextField(), 1, 1);
grid.getChildren().addAll(
    new Label("Name:"), new TextField(),
    new Label("Email:"), new TextField()
);
```

6.2 JavaFX UI Controls

```
Label label = new Label("Enter Name:");
label.setFont(new Font("Arial", 14));

TextField tf = new TextField();
tf.setPromptText("Enter your name"); // placeholder
String text = tf.getText();

Button btn = new Button("Submit");
btn.setOnAction(e -> System.out.println("Submitted"));

RadioButton rb1 = new RadioButton("Male");
RadioButton rb2 = new RadioButton("Female");
ToggleGroup tg = new ToggleGroup();
rb1.setToggleGroup(tg);
rb2.setToggleGroup(tg);

CheckBox cb1 = new CheckBox("Java");
CheckBox cb2 = new CheckBox("Python");
cb1.setSelected(true);

Hyperlink link = new Hyperlink("Click here");
link.setOnAction(e -> System.out.println("Link clicked"));

MenuBar menuBar = new MenuBar();
Menu fileMenu = new Menu("File");
MenuItem openItem = new MenuItem("Open");
openItem.setOnAction(e -> System.out.println("Open"));
fileMenu.getItems().add(openItem);
menuBar.getMenus().add(fileMenu);
```

```

Tooltip tooltip = new Tooltip("This is a button");
btn.setTooltip(tooltip);

FileChooser fc = new FileChooser();
fc.setTitle("Select File");
fc.getExtensionFilters().add(
    new FileChooser.ExtensionFilter("Text Files", "*.txt"));
File selected = fc.showOpenDialog(stage);

```

CSS Styling in JavaFX:

```

Button btn = new Button("Styled Button");
btn.setStyle("-fx-background-color: blue; -fx-text-fill: white; " +
    "-fx-font-size: 14px; -fx-padding: 10 20;");

```

Or use external stylesheet: `scene.getStylesheets().add("style.css");`

Unit 7: Servlets and JSP

7.1 Servlets

Web Container (Servlet Container): - Manages servlet lifecycle - Handles HTTP requests/responses - Provides: threading, security, session management, JSP support - Examples: Apache Tomcat, Jetty, GlassFish

Servlet Life Cycle:

1. Loading – Container loads servlet **class**
2. Instantiation – Constructor called
3. Initialization – `init()` called once
4. Request Handling – `service()` called **for** each request
 - └─ `doGet()` – GET requests
 - └─ `doPost()` – POST requests
5. Destruction – `destroy()` called once before unloading

```

@WebServlet("/hello") // annotation – no web.xml needed
public class HelloServlet extends HttpServlet {

    @Override
    public void init() throws ServletException {
        // Called once – for setup (DB connections, etc.)
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        resp.setContentType("text/html");
        PrintWriter out = resp.getWriter();
        out.println("<h1>Hello from Servlet</h1>");

        // Reading form parameters
        String name = req.getParameter("name");
        out.println("<p>Hello, " + name + "</p>");
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        // Handle POST requests
        doGet(req, resp); // or handle differently
    }

    @Override
    public void destroy() {
        // Called once – cleanup resources
    }
}

```

Servlet API Hierarchy:

```

javax.servlet.Servlet (interface)
└─ GenericServlet (abstract)

```

```

└─ HttpServlet (abstract – for HTTP)
    └─ YourServlet

```

GET vs POST:

GET	POST
Parameters in URL	Parameters in request body
Limited length (~2KB)	Unlimited
Bookmarkable	Not bookmarkable
Visible in browser history	Not visible
Less secure	More secure (data in body)
Idempotent	Not idempotent
<code>doGet()</code>	<code>doPost()</code>

Processing Forms:

```

// HTML form
<form action="login" method="post">
    Username: <input type="text" name="username"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Login">
</form>

// Servlet
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        String username = req.getParameter("username");
        String password = req.getParameter("password");
        // Validate and respond
    }
}

```

Database Access with Servlets:

```

@WebServlet("/register")
public class RegisterServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) {
        String name = req.getParameter("name");
        try (Connection conn = DriverManager.getConnection(url, user, pass);
            PreparedStatement pstmt = conn.prepareStatement(
                "INSERT INTO users (name) VALUES (?)")) {
            pstmt.setString(1, name);
            int rows = pstmt.executeUpdate();
            resp.getWriter().println("Registered: " + rows + " row(s)");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

Handling Cookies:

```

// Creating cookie
Cookie cookie = new Cookie("username", "ram");
cookie.setMaxAge(60 * 60 * 24); // 1 day in seconds
resp.addCookie(cookie);

// Reading cookies
Cookie[] cookies = req.getCookies();
if (cookies != null) {
    for (Cookie c : cookies) {
        if (c.getName().equals("username")) {
            String val = c.getValue();
        }
    }
}

// Deleting cookie
Cookie cookie = new Cookie("username", "");

```



```
cookie.setMaxAge(0); // immediate expiry
resp.addCookie(cookie);
```

Handling Session:

```
// Get or create session
HttpSession session = req.getSession(true);

// Store data
session.setAttribute("user", username);

// Retrieve data
String user = (String) session.getAttribute("user");

// Remove data
session.removeAttribute("user");

// Invalidate entire session
session.invalidate();

// Session timeout (in web.xml)
// <session-config><session-timeout>30</session-timeout></session-config>

// URL rewriting (when cookies disabled)
String url = resp.encodeURL("/nextpage.jsp");
```

7.2 JSP (Java Server Pages)

Servlet vs JSP:

Servlet	JSP
Java code with embedded HTML	HTML with embedded Java
Better for business logic	Better for presentation
doGet() / doPost() methods	Scriptlets, expressions, tags
Harder to maintain (HTML in strings)	Easier to maintain
Controller in MVC	View in MVC

JSP Access Model:

Browser → JSP → Servlet Container → JSP Engine → Translated to Servlet → Compiled → Executed

JSP Syntax:

```
<%-- JSP Comment (not sent to client) --%>

<!-- Directive -- page configuration -->
<%@ page language="java" contentType="text/html" %>
<%@ page import="java.util.*, java.sql.*" %>
<%@ page errorPage="error.jsp" %>
<%@ page isErrorPage="true" %>
<%@ page session="true" %>
<%@ page buffer="8kb" %>

<%@ include file="header.jsp" %> <!-- Include directive -- compile-time include -->
<jsp:include page="footer.jsp" /> <!-- Include action -- runtime include -->

<!-- Declaration -- class-level members -->
<%! int counter = 0; %>
<%! public String greet(String name) { return "Hello " + name; } %>

<!-- Expression -- prints to output -->
<%= new java.util.Date() %>
<%= counter %>
<%= greet("Ram") %>

<!-- Scriptlet -- Java code block -->
<%
String name = request.getParameter("name");
if (name != null) {
    out.println("<h2>Welcome, " + name + "</h2>");
}
%>
```

```

    for (int i = 0; i < 5; i++) {
        out.println("<p>Line " + i + "</p>");
    }
%>

<!-- jsp:useBean - JavaBeans integration -->
<jsp:useBean id="user" class="com.example.User" scope="session" />
<jsp:setProperty name="user" property="name" value="Ram" />
<jsp:getProperty name="user" property="name" />

```

JSP Implicit Objects (9 objects automatically available):

Object	Type	Description
request	HttpServletRequest	Client request (form data, headers)
response	HttpServletResponse	Server response
session	HttpSession	User session (per-user data)
application	ServletContext	Application-wide (shared across all users)
out	JspWriter	Output stream to client
pageContext	PageContext	Page context (access to all scopes)
config	ServletConfig	Servlet configuration
page	Object	Reference to current JSP page (this)
exception	Throwable	Only available in <code>isErrorPage="true"</code>

Object Scope (4 scopes — from narrowest to widest):

Scope	Duration	Access
page	Single page request	<code>pageContext.setAttribute("k", v)</code>
request	Single request (forwarded pages)	<code>request.setAttribute("k", v)</code>
session	User session (multiple requests)	<code>session.setAttribute("k", v)</code>
application	Entire web application	<code>application.setAttribute("k", v)</code>

Processing Forms with JSP:

```

<%-- Input form --%>
<form action="result.jsp" method="post">
    First Name: <input type="text" name="fname"><br>
    Last Name: <input type="text" name="lname"><br>
    <input type="submit" value="Submit">
</form>

<%-- result.jsp --%>
<%
    String fname = request.getParameter("fname");
    String lname = request.getParameter("lname");
    out.println("Full Name: " + fname + " " + lname);
%>

```

Database Access with JSP:

```

<%@ page import="java.sql.*" %>
<%
    String url = "jdbc:mysql://localhost:3306/mydb";
    try (Connection conn = DriverManager.getConnection(url, "root", "pass");
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM student")) {
        while (rs.next()) {
            out.println(rs.getInt("id") + ": " + rs.getString("name") + "<br>");
        }
    } catch (SQLException e) {
        out.println("Error: " + e.getMessage());
    }
%>

```

7.3 Java Web Frameworks (Introduction)

- **Spring MVC** — Most popular. Annotations, DI, REST support
- **Struts 2** — Apache framework, action-based
- **JavaServer Faces (JSF)** — Component-based, MVC

- **Hibernate** — ORM (Object-Relational Mapping) — often paired with Spring

Key concepts: Front Controller (DispatcherServlet), Dependency Injection, Inversion of Control, ORM, REST APIs

Unit 8: RMI and CORBA

8.1 RMI (Remote Method Invocation)

Introduction — RMI allows a Java object to invoke methods on a remote Java object as if it were local.

RMI Architecture Layers:

```
Client → Stub → Remote Reference Layer → Transport Layer → Server
                                     ↓
Server → Skeleton → Remote Reference Layer → Transport Layer → Client
```

1. **Stub (client-side):** Proxy object — looks like the remote object to the client. Marshals parameters and unmarshals return values.
2. **Skeleton (server-side):** Receives remote call, unmarshals params, invokes actual method, marshals result (Java 2+ uses reflection instead of skeleton).
3. **Remote Reference Layer:** Handles references, multicast, etc.
4. **Transport Layer:** Manages TCP connections.

Creating an RMI Application — 5 Steps:

```
// Step 1: Define Remote Interface
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
}

// Step 2: Implement Remote Interface
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator {
    protected CalculatorImpl() throws RemoteException { }

    public int add(int a, int b) { return a + b; }
    public int multiply(int a, int b) { return a * b; }
}

// Step 3: Create Server (RMI Registry)
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class RMIServer {
    public static void main(String[] args) throws Exception {
        Calculator calc = new CalculatorImpl();
        Registry registry = LocateRegistry.createRegistry(1099); // default RMI port
        registry.rebind("CalculatorService", calc);
        System.out.println("Server is ready");
    }
}

// Step 4: Create Client
import java.rmi.registry.LocateRegistry;

public class RMIClient {
    public static void main(String[] args) throws Exception {
        Registry registry = LocateRegistry.getRegistry("localhost", 1099);
        Calculator calc = (Calculator) registry.lookup("CalculatorService");

        int result = calc.add(10, 20); // remote method call!
        System.out.println("Result: " + result);
    }
}

// Step 5: Compile and Run
// javac *.java
// rmiregistry & // start RMI registry
```

```
// java RMIServer &           // start server
// java RMIClient             // run client
```

RMI key classes/interfaces: - `java.rmi.Remote` — marker interface for remote objects - `java.rmi.server.UnicastRemoteObject` — exports remote object, generates stub - `java.rmi.registry.LocateRegistry` — gets/creates registry - `java.rmi.Naming` — binds/looks up (older API, `Registry` preferred)

8.2 CORBA (Common Object Request Broker Architecture)

Introduction — CORBA is middleware that allows objects written in different languages (Java, C++, Python) to communicate over a network.

RMI vs CORBA:

RMI	CORBA
Java only	Language-independent
No IDL needed	Uses IDL (Interface Definition Language)
Simpler	More complex
RMI Registry	ORB (Object Request Broker)
Tightly coupled to Java	Loosely coupled
Uses Java serialization	Uses IIOP protocol

CORBA Architecture:

```
Client → Stub → ORB → IIOP → ORB → Skeleton → Servant (Implementation)
      ↑ Network ↓
```

- **ORB (Object Request Broker):** Middleware that handles communication — finds object, delivers request, returns result
- **Stub:** Client-side proxy (generated from IDL)
- **Skeleton:** Server-side handler (generated from IDL)
- **IIOP (Internet Inter-ORB Protocol):** Communication protocol
- **POA (Portable Object Adapter):** Manages object lifecycle

IDL (Interface Definition Language):

```
// calculator.idl
module CalculatorApp {
    interface Calculator {
        long add(in long a, in long b);
        long multiply(in long a, in long b);
        oneway void shutdown();
    };
};
```

IDL data types: long, short, float, double, char, boolean, string, struct, sequence, array, enum

Parameter passing: `in` (input), `out` (output), `inout` (input+output)

Simple CORBA Program Steps:

1. Write IDL file (calculator.idl)
2. Run idlj compiler: `idlj -fall calculator.idl`
→ Generates: `Calculator.java`, `CalculatorOperations.java`, `_CalculatorStub.java`, `CalculatorPOA.java`, etc.
3. Implement servant (**extends** `CalculatorPOA`)
4. Create server (initialize ORB, create POA, activate object)
5. Create client (initialize ORB, narrow reference, call methods)

Example (after idlj generation):

```
// Servant Implementation
class CalculatorImpl extends CalculatorPOA {
    public int add(int a, int b) { return a + b; }
    public int multiply(int a, int b) { return a * b; }
    public void shutdown() { /* cleanup */ }
}

// Server
ORB orb = ORB.init(args, null);
POA rootPOA = POAHelper.narrow(orb.resolve_initial_references("RootPOA"));
rootPOA.the_POAManager().activate();

CalculatorImpl impl = new CalculatorImpl();
org.omg.CORBA.Object ref = rootPOA.servant_to_reference(impl);
Calculator href = CalculatorHelper.narrow(ref);
```

```
orb.run();

// Client
ORB orb = ORB.init(args, null);
org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");
Calculator calc = CalculatorHelper.narrow(objRef);
int result = calc.add(10, 20);
```

Quick Tips for Exam: - **10-mark questions:** Usually combine 2+ topics with a program. Expect: Swing+Events, Servlets+JSP+DB, Multithreading+File I/O, RMI application - **5-mark questions:** Single topic — theory or short program. Cover all units. - **Code snippets:** Write clean, compilable-looking code. Add comments for key lines. - **Diagrams:** Always draw when asked (JVM architecture, JDBC architecture, RMI layers, CORBA architecture, Thread states) — they carry marks.